

Python numeric types

Technical Notes

Rodrigo Kang

Table of contents

1	Boolean Values and Propositional Logic	2
1.1	Logical Operators and Truth Tables	3
1.1.1	Negation	4
1.1.2	Conjunction	4
1.1.3	Disjunction	5
1.2	Comparison Operators	6
2	Integers and Floating-Point Numbers	7
2.1	Integers	8
2.2	Floating-Point Numbers	9
2.3	Arithmetic Operations	10
3	Decimal, Binary, and Positional Notation	12
3.1	Floating-Point Representation	15
3.2	Floating-Point Arithmetic	17
4	Complex Numbers	19
4.1	Complex Arithmetic	20
4.2	The cmath Module	22
5	Problems	24
5.1	Problem 1 — Boolean expressions	24
5.2	Problem 2 — Operator precedence	24
5.3	Problem 3 — Comparisons	25
5.4	Problem 4 — Compound conditions	25
5.5	Problem 5 — Chained comparisons	25
5.6	Problem 6 — Logical equivalence	26
5.7	Problem 7 — Arithmetic operators	26
5.8	Problem 8 — Floor division	26
5.9	Problem 9 — Division algorithm	27
5.10	Problem 10 — Integer conversion	27
5.11	Positional and Binary Representation	27
5.12	Problem 11 — Decimal to binary	27

5.13	Problem 12 — Binary to decimal	28
5.14	Problem 13 — Multiple representations	28
5.15	Problem 14 — Fractional binary representation	28
5.16	Problem 15 — Decimal fractions in binary	28
5.17	Problem 16 — The 0.1 + 0.2 problem	28
5.18	Problem 17 — Exact and inexact representations	29
5.19	Problem 18 — Floating-point equality	29
5.20	Problem 19 — Machine epsilon	30
5.21	Problem 20 — Precision depends on magnitude	30
5.22	Problem 21 — Accumulation error	30
5.23	Problem 22 — Summation accuracy	31
5.24	Problem 23 — Catastrophic cancellation	31
5.25	Problem 24 — Stable computation	31
5.26	Problem 25 — Creating complex numbers	32
5.27	Problem 26 — Complex arithmetic	32
5.28	Problem 27 — math versus cmath	32
5.29	Problem 28 — Cartesian and polar representations	33
5.30	Problem 29 — Euler’s identity	33
5.31	Problem 30 — Predict the output	33
5.32	Problem 31 — Equality across numeric types	34
5.33	Problem 32 — A subtle dictionary key	34
5.34	Problem 33 — Floating-point threshold	34
5.35	Problem 34 — Comparing model metrics	35
5.36	Problem 35 — Large integer versus floating point	35
5.37	Problem 36 — Numerical preprocessing	35
5.38	Problem 37 — Probability normalization	36
5.39	Problem 38 — Debugging a numerical comparison	36
5.40	Problem 39 — Reasoning about numerical stability	36
5.41	Problem 40 — Choosing a numeric representation	36

1 Boolean Values and Propositional Logic

Many computational decisions can be expressed in terms of two logical states: **true** or **false**. Python represents these states through the Boolean type, `bool`, whose two possible values are `True` and `False`.

Boolean values provide the basis for conditions, comparisons, logical expressions, and control flow. Their conceptual foundation can be understood through **propositional logic**, the formal system used to reason about statements and their truth values.

A **proposition** is a declarative statement to which a definite truth value can be assigned. Propositions are commonly represented symbolically by letters such as p , q , and r . For example:

- “7 is a prime number” is a true proposition.
- “ $2 + 3 = 6$ ” is a false proposition.

The essential characteristic is that each proposition is assigned one of two truth values: **true** or **false**. Boolean logic provides operations for combining and transforming these truth values, forming the logical basis of conditional computation.

In Python, a Boolean value can be assigned directly:

```
1 p = True
2
3 type(p)
```

bool

The built-in `type()` function returns the type of an object. In this case, `p` refers to an object of type `bool`.

```
1 q = False
2
3 type(q)
```

bool

An implementation detail worth noting is that Python's `bool` type is a subclass of `int`. Consequently, `True` and `False` behave numerically as 1 and 0 in appropriate contexts:

```
1 print(int(True))
2 print(int(False))
3
4 print(True == 1)
5 print(False == 0)
```

```
1
0
True
True
```

Despite this relationship, Boolean values should generally be interpreted according to their logical meaning rather than treated simply as integers.

1.1 Logical Operators and Truth Tables

Propositional logic defines operations that transform or combine truth values. Three fundamental operations correspond directly to Python's `not`, `and`, and `or` operators.

1.1.1 Negation

The **negation** of a proposition p , denoted $\neg p$, reverses its truth value.

p	$\neg p$
True	False
False	True

Table 1: Truth table for logical negation.

Python represents logical negation with the not operator:

```
1 p = True
2
3 not p
```

False

```
1 q = False
2
3 not q
```

True

1.1.2 Conjunction

The **conjunction** of two propositions, denoted $p \wedge q$, is true only when both propositions are true.

p	q	$p \wedge q$
True	True	True
True	False	False
False	True	False
False	False	False

Table 2: Truth table for logical conjunction.

Python represents conjunction with the and operator:

```

1 p = True
2 q = True
3
4 p and q

```

True

Logical expressions can be combined:

```

1 p and (not q)

```

False

1.1.3 Disjunction

The **disjunction** of two propositions, denoted $p \vee q$, is true when at least one of the propositions is true.

p	q	$p \vee q$
True	True	True
True	False	True
False	True	True
False	False	False

Table 3: Truth table for logical disjunction.

Python represents disjunction with the or operator:

```

1 p = False
2 q = False
3
4 p or q

```

False

```

1 (not p) or q

```

True

These operators provide the logical machinery required to construct more complex conditions and decision rules.

1.2 Comparison Operators

Boolean values are frequently produced by **comparison expressions**. Python provides several comparison operators:

Operator	Meaning
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
==	Equal to
!=	Not equal to

Table 4: Comparison operators in Python.

Each comparison evaluates to a Boolean value:

```
1 print(7 == 7.0)
2 print(3.14 > 9)
3 print(7 != "7")
4 print(0.01 <= 1)
```

True
False
True
True

Python considers 7 and 7.0 numerically equal even though they have different types:

```
1 print(7 == 7.0)
2 print(type(7))
3 print(type(7.0))
```

```
True
<class 'int'>
<class 'float'>
```

By contrast, the integer 7 and the string "7" represent different kinds of information and do not compare as equal:

```
1 7 == "7"
```

False

Comparisons can be combined with logical operators to construct compound conditions:

```
1 age = 17
2
3 (age >= 16) and (age <= 40)
```

True

Python also supports **chained comparisons**, providing a more concise representation of the same condition:

```
1 16 <= age <= 40
```

True

Both expressions evaluate whether age lies between the specified limits.

2 Integers and Floating-Point Numbers

Numerical computation requires a representation of mathematical quantities within the finite resources of a computer.

Consider the area of a circle:

$$A = \pi r^2. \tag{1}$$

Here, r represents the radius and π is an irrational number whose decimal expansion begins with 3.14159 ... and continues indefinitely without repeating.

A computer, however, has finite memory and cannot store an arbitrary real number with infinitely many digits. This introduces a fundamental distinction between **mathematical numbers** and their **computational representations**.

Python provides two principal built-in types for ordinary real-valued numerical computation:

- int for integers;
- float for floating-point numbers.

Python also provides the complex type for complex-valued computation, which will be considered separately.

2.1 Integers

The int type represents integer values:

```
1 x = 5
2
3 type(x)
```

```
int
```

Integers may be positive, negative, or zero:

```
1 a = 100
2 b = -25
3 c = 0
4
5 print(type(a))
6 print(type(b))
7 print(type(c))
```

```
<class 'int'>
<class 'int'>
<class 'int'>
```

Python integers support arbitrary precision: their size is not restricted to a fixed 32-bit or 64-bit range. In practice, the maximum size of an integer is constrained primarily by available memory.

2.2 Floating-Point Numbers

The float type represents floating-point numbers:

```
1 x = 3.14
2
3 type(x)
```

float

A number containing a decimal point is interpreted as a float:

```
1 print(type(5))
2 print(type(5.0))
```

<class 'int'>
<class 'float'>

Python also accepts a trailing decimal point:

```
1 print(type(3.0))
2 print(type(3.))
```

<class 'float'>
<class 'float'>

Both 3.0 and 3. represent the same floating-point value.

The int() and float() constructors can be used for explicit numeric conversion:

```
1 print(float(3))
2 print(int(7.0))
```

3.0
7

Converting an integer to a floating-point number preserves its numerical value for integers that can be represented exactly within the available floating-point precision:

```
1 float(7)
```

7.0

Converting a floating-point number to an integer discards the fractional part by truncating toward zero:

```
1 print(int(3.5))  
2 print(int(-3.5))
```

3
-3

This is not equivalent to rounding to the nearest integer.

2.3 Arithmetic Operations

Python provides the standard arithmetic operators for numerical computation:

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	True division
//	Floor division
%	Modulo
**	Exponentiation

Table 5: Arithmetic operators in Python.

For example:

```
1 x = 7  
2 y = 3  
3  
4 print(x + y)  
5 print(x - y)
```

```
6 print(x * y)
7 print(x / y)
8 print(x // y)
9 print(x % y)
10 print(x ** y)
```

```
10
4
21
2.3333333333333335
2
1
343
```

The / operator performs **true division** and returns a floating-point result:

```
1 8 / 2
```

```
4.0
```

even when the mathematical result is an integer.

The // operator performs **floor division**:

```
1 7 // 2
```

```
3
```

It is important to distinguish floor division from simple truncation toward zero, particularly for negative values:

```
1 -7 // 2
```

```
-4
```

The result is -4, because floor division returns the floor of the quotient.

The % operator returns the corresponding remainder:

```
1 7 % 3
```

1

Floor division and modulo satisfy the relationship

$$x = (x // y) y + (x \% y) . \quad (2)$$

Exponentiation is performed with **:

```
1 2 ** 3
```

8

Python also provides the built-in pow() function:

```
1 pow(2, 3)
```

8

For ordinary two-argument use, both expressions compute 2^3 .

3 Decimal, Binary, and Positional Notation

Understanding numerical representation begins with the **positional numeral system**. In a positional system, the value represented by a digit depends not only on the digit itself but also on its position within the numeral.

The number of distinct digits available in a positional numeral system determines its **base**, conventionally denoted by b . Common systems include:

Base	Number system	Digits
2	Binary	0, 1
8	Octal	0–7
10	Decimal	0–9
16	Hexadecimal	0–9, A–F

Table 6: Common positional numeral systems.

For a base b , a numeral with digits

$$d_n d_{n-1} \dots d_2 d_1 d_0$$

represents the value

$$\sum_{k=0}^n d_k b^k, \quad (3)$$

where each digit satisfies

$$0 \leq d_k < b.$$

For example, the decimal number 472 can be written as

$$472 = 4 \times 10^2 + 7 \times 10^1 + 2 \times 10^0. \quad (4)$$

The same principle applies in binary. The binary numeral 1011_2 represents

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 11_{10}. \quad (5)$$

Binary representation is particularly important in computing because digital systems ultimately encode information using two distinguishable states, conventionally represented as 0 and 1.

Python allows integer literals to be expressed directly in several bases:

```
1 decimal_value = 11
2 binary_value = 0b1011
3 octal_value = 0o13
4 hexadecimal_value = 0xB
5
6 print(decimal_value)
7 print(binary_value)
8 print(octal_value)
9 print(hexadecimal_value)
```

```
11
11
11
11
```

Although these literals use different numeral systems in the source code, they represent the same integer value:

```
1 decimal_value == binary_value == octal_value == hexadecimal_value
```

True

Python also provides built-in functions for obtaining string representations of integers in different bases:

```
1 x = 11
2
3 print(bin(x))
4 print(oct(x))
5 print(hex(x))
```

0b1011

0o13

0xb

The positional principle extends naturally to fractional values. Digits to the right of the radix point correspond to negative powers of the base. In decimal notation, for example,

$$0.125 = 1 \times 10^{-1} + 2 \times 10^{-2} + 5 \times 10^{-3}. \quad (6)$$

In binary,

$$0.101_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.625_{10}. \quad (7)$$

This extension from integer to fractional positional representation provides the foundation for understanding how real-valued quantities can be approximated computationally. Because a computer has finite storage, not every real number—and not even every finite decimal fraction—can be represented exactly using a finite binary expansion. This limitation leads directly to **floating-point representation and floating-point arithmetic**.

3.1 Floating-Point Representation

A computer can store only a finite amount of information. Consequently, while integers can be represented exactly within the available computational resources, arbitrary real numbers generally cannot. Many real numbers require infinitely many digits, and even some numbers with finite decimal representations have infinite representations in binary.

For example, the decimal fraction 0.125 has the finite binary representation

$$0.125_{10} = 0.001_2, \quad (8)$$

because

$$0.001_2 = 1 \times 2^{-3} = \frac{1}{8}.$$

By contrast, the decimal number 0.1 cannot be represented by a finite binary fraction. Its binary expansion continues indefinitely:

$$0.1_{10} = 0.00011001100110011 \dots_2. \quad (9)$$

A finite computer representation must therefore approximate such a value.

Floating-point representation addresses this problem using an idea analogous to **scientific notation**. In decimal scientific notation, a number can be represented as a significand multiplied by a power of the base. For example,

$$12345 = 1.2345 \times 10^4. \quad (10)$$

The same principle can be applied in binary. A non-zero binary number can be expressed in normalized form as

$$(-1)^s \times m \times 2^e, \quad (11)$$

where:

- s determines the sign;
- m is the **significand**;
- e is the **exponent**.

The term **mantissa** is also commonly encountered, although *significand* is the more precise term in the context of floating-point representation.

Most modern computers follow the **IEEE 754** standard for floating-point arithmetic. Python’s built-in float type is typically implemented using IEEE 754 **binary64**, commonly known as double precision.

A binary64 floating-point value occupies 64 bits, divided conceptually into three components:

Component	Bits	Purpose
Sign	1	Determines whether the number is positive or negative
Exponent	11	Determines the scale of the number
Fraction	52	Stores the significant binary digits

Table 7: Main components of an IEEE 754 binary64 floating-point value.

For normal finite values, the represented number has the form

$$(-1)^s \times (1.f)_2 \times 2^{e-1023}, \quad (12)$$

where f represents the stored fractional bits and 1023 is the exponent bias used by binary64.

Because the leading binary digit of a normalized significand is implicitly 1, binary64 effectively provides **53 bits of significand precision**, even though only 52 fraction bits are explicitly stored.

This corresponds to approximately 15–17 significant decimal digits of precision.

Python exposes information about the floating-point implementation through `sys.float_info`:

```
1 import sys
2
3 sys.float_info
```

```
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-308)
```

Some particularly useful properties are:

```
1 print(sys.float_info.max)
2 print(sys.float_info.min)
3 print(sys.float_info.epsilon)
```



```
1.7976931348623157e+308
2.2250738585072014e-308
2.220446049250313e-16
```

Here, max represents the largest finite positive floating-point value, min represents the smallest positive **normal** floating-point value, and epsilon represents the difference between 1.0 and the next larger representable floating-point number.

An important distinction is therefore required between **range** and **precision**. Floating-point representation can describe numbers across an extremely large range of magnitudes, but only with a finite number of significant digits.

3.2 Floating-Point Arithmetic

Because many real numbers cannot be represented exactly in binary floating-point form, arithmetic operations are performed on finite approximations. The result of an operation may itself need to be rounded to the nearest representable floating-point value.

A familiar example is:

```
1 0.1 + 0.2

0.30000000000000004
```

Mathematically,

$$0.1 + 0.2 = 0.3.$$

However, Python evaluates:

```
1 0.1 + 0.2 == 0.3
```

False

as False.

This does not indicate an error in Python's arithmetic. The values 0.1, 0.2, and 0.3 cannot be represented exactly as finite binary fractions. Python instead stores nearby representable values, and the result of the arithmetic reflects those approximations.

The difference can be inspected explicitly:

```
1 (0.1 + 0.2) - 0.3
```

```
5.551115123125783e-17
```

For this reason, direct equality comparisons between floating-point results are often inappropriate when values have been obtained through numerical computation.

Python's math module provides `math.isclose()` for approximate comparison:

```
1 import math
2
3 math.isclose(0.1 + 0.2, 0.3)
```

```
True
```

The comparison is based on relative and absolute tolerances. Conceptually, two values a and b are considered sufficiently close when

$$|a - b| \leq \max(\text{rel_tol} \max(|a|, |b|), \text{abs_tol}). \quad (13)$$

The tolerances can be specified explicitly when the numerical context requires them:

```
1 math.isclose(
2     0.1 + 0.2,
3     0.3,
4     rel_tol=1e-9,
5     abs_tol=0.0
6 )
```

```
True
```

The appropriate tolerance depends on the scale of the quantities involved and on the requirements of the application; there is no universal tolerance suitable for every numerical problem.

Floating-point errors can also accumulate across repeated operations. For example:

```
1 total = 0.0
2
3 for _ in range(10):
4     total += 0.1
5
6 total
```

0.9999999999999999

The mathematical result is exactly 1, but the floating-point result may differ slightly because each operation is performed using finite approximations.

Numerical error is not limited to the representation of individual values. Certain computational procedures can amplify small rounding errors. One important example is **catastrophic cancellation**, which can occur when subtracting two nearly equal floating-point numbers: significant leading digits cancel, leaving a result whose relative accuracy may be substantially reduced.

These considerations lead to the broader concept of **numerical stability**. A numerically stable procedure is designed so that unavoidable representation and rounding errors do not grow excessively during computation.

For Data Science and Machine Learning, the practical implication is not that floating-point arithmetic is unreliable. It is the standard foundation of numerical computation. Rather, numerical results should be interpreted with an understanding of finite precision, appropriate comparison tolerances, and the numerical behaviour of the algorithms being used.

4 Complex Numbers

The real numbers are sufficient for many computational tasks, but some mathematical and scientific problems require an extension to the **complex numbers**.

A complex number has the form

$$z = a + bi, \tag{14}$$

where a and b are real numbers and i is the imaginary unit satisfying

$$i^2 = -1. \tag{15}$$

The value a is the **real part** of z , while b is its **imaginary part**.

Python provides the built-in complex type for representing complex numbers. Python uses j rather than i for the imaginary unit:

```
1 z = 3 + 4j
2
3 type(z)
```

complex

The use of j is conventional in electrical engineering and avoids potential ambiguity with identifiers commonly represented by i .

The real and imaginary components can be accessed through the `.real` and `.imag` attributes:

```
1 print(z.real)
2 print(z.imag)
```

```
3.0
4.0
```

Both components are represented as floating-point values:

```
1 print(type(z.real))
2 print(type(z.imag))
```

```
<class 'float'>
<class 'float'>
```

Complex numbers can also be created explicitly using the `complex()` constructor:

```
1 z = complex(3, 4)
2
3 z
```

```
(3+4j)
```

which represents

$$z = 3 + 4i.$$

4.1 Complex Arithmetic

Python supports the standard arithmetic operations for complex numbers:

```

1 z1 = 3 + 4j
2 z2 = 1 - 2j
3
4 print(z1 + z2)
5 print(z1 - z2)
6 print(z1 * z2)
7 print(z1 / z2)

```

```

(4+2j)
(2+6j)
(11-2j)
(-1+2j)

```

The **complex conjugate** of

$$z = a + bi$$

is

$$\bar{z} = a - bi. \quad (16)$$

Python provides the `.conjugate()` method:

```

1 z = 3 + 4j
2
3 z.conjugate()

```

```

(3-4j)

```

The magnitude, or modulus, of a complex number is

$$|z| = \sqrt{a^2 + b^2}. \quad (17)$$

For example,

$$|3 + 4i| = 5.$$

Python's built-in `abs()` function computes this magnitude:

```
1 z = 3 + 4j
2
3 abs(z)
```

5.0

4.2 The cmath Module

The standard math module is designed primarily for real-valued mathematical operations. Python provides the cmath module for mathematical functions defined over complex numbers.

For example, attempting to compute the real square root of a negative number with `math.sqrt()` is outside the real domain, whereas `cmath.sqrt()` returns the corresponding complex value:

```
1 import cmath
2
3 cmath.sqrt(-1)
```

1j

The module provides complex-valued versions of functions such as square roots, exponentials, logarithms, and trigonometric functions.

For example:

```
1 import cmath
2
3 z = 1 + 1j
4
5 print(cmath.exp(z))
6 print(cmath.log(z))
7 print(cmath.sin(z))
```

```
(1.4686939399158851+2.2873552871788423j)
(0.34657359027997264+0.7853981633974483j)
(1.2984575814159773+0.6349639147847361j)
```

Complex numbers can also be represented in **polar form**. If

$$z = a + bi,$$

then

$$z = r(\cos \theta + i \sin \theta), \quad (18)$$

where

$$r = |z|$$

is the magnitude and θ is the phase angle.

The `cmath.polar()` function converts a complex number to its magnitude and phase:

```
1 z = 3 + 4j
2
3 cmath.polar(z)
```

```
(5.0, 0.9272952180016122)
```

Conversely, `cmath.rect()` constructs a complex number from polar coordinates:

```
1 r, theta = cmath.polar(z)
2
3 cmath.rect(r, theta)
```

```
(3.0000000000000004+3.9999999999999996j)
```

Euler's identity provides an important connection between complex exponentials and trigonometric functions:

$$e^{i\pi} + 1 = 0. \quad (19)$$

It can be evaluated numerically with:

```
1 cmath.exp(1j * cmath.pi) + 1
```

```
1.2246467991473532e-16j
```

The result may contain a very small imaginary component rather than being represented as exactly zero. This is another consequence of the finite-precision floating-point arithmetic discussed previously.

5 Problems

The following problems review the main concepts introduced in this section and provide practice with Python's Boolean and numerical types, logical reasoning, numerical representation, and floating-point arithmetic.

5.1 Problem 1 — Boolean expressions

Without executing the code, determine the value and type of each expression. Then verify your answers in Python.

```
1 True and False
2 True or False
3 not True
4 not (True and False)
5 (True or False) and not False
```

5.2 Problem 2 — Operator precedence

Without executing the code, determine the result of each expression:

```
1 True or False and False
2 (True or False) and False
3 not True or True
4 not (True or True)
```

Explain how operator precedence affects the results.

5.3 Problem 3 — Comparisons

Determine whether each expression evaluates to True or False:

```
1 7 == 7.0
2 7 == "7"
3 True == 1
4 False == 0
5 3 < 5 < 10
6 3 < 5 > 10
```

Explain any results that may initially appear surprising.

5.4 Problem 4 — Compound conditions

Given:

```
1 age = 32
2 experience = 5
3 has_degree = True
```

Write a Boolean expression that evaluates to True only when a candidate:

- is at least 25 years old;
- has at least 3 years of experience; and
- either has a degree or has at least 7 years of experience.

5.5 Problem 5 — Chained comparisons

Rewrite the following expression using Python's chained comparison syntax:

```
1 x >= 0 and x <= 1
```

Then write a condition that determines whether a value lies in the interval

$$-1 < x \leq 1.$$

5.6 Problem 6 — Logical equivalence

For all possible Boolean values of p and q , compare:

```
1 not (p and q)
```

with:

```
1 (not p) or (not q)
```

Then compare:

```
1 not (p or q)
```

with:

```
1 (not p) and (not q)
```

Determine whether each pair is logically equivalent.

5.7 Problem 7 — Arithmetic operators

Without executing the code, determine the result and type of each expression:

```
1 7 + 3
2 7 - 3
3 7 * 3
4 7 / 3
5 7 // 3
6 7 % 3
7 7 ** 3
```

Then verify your answers in Python.

5.8 Problem 8 — Floor division

Predict the result of:

```
1 7 // 2
2 -7 // 2
3 7 // -2
4 -7 // -2
```

Explain why floor division with negative numbers may differ from truncation toward zero.

5.9 Problem 9 — Division algorithm

For several positive and negative integer values of x and non-zero values of y , verify computationally that:

$$x = (x//y)y + (x\%y).$$

Investigate how the sign of the remainder behaves when y is negative.

5.10 Problem 10 — Integer conversion

Without executing the code, determine the result of:

```
1 int(3.9)
2 int(-3.9)
3 float(7)
4 int(True)
5 int(False)
```

Explain the difference between integer conversion and rounding.

5.11 Positional and Binary Representation

5.12 Problem 11 — Decimal to binary

Convert the following decimal integers to binary by hand:

13, 42, 100.

Verify your results using Python's `bin()` function.

5.13 Problem 12 — Binary to decimal

Convert the following binary numbers to decimal without using Python:

$$10101_2, \quad 111111_2, \quad 10000001_2.$$

Verify the results computationally.

5.14 Problem 13 — Multiple representations

Represent the decimal integer 255 in:

- binary;
- octal;
- hexadecimal.

Verify that all representations correspond to the same Python integer.

5.15 Problem 14 — Fractional binary representation

Convert the following finite binary fractions to decimal:

$$0.1_2, \quad 0.01_2, \quad 0.101_2, \quad 0.111_2.$$

5.16 Problem 15 — Decimal fractions in binary

Determine which of the following decimal fractions have finite binary representations:

$$0.5, \quad 0.25, \quad 0.125, \quad 0.1, \quad 0.2, \quad 0.75.$$

Explain the mathematical property that determines whether a rational number has a finite binary representation.

5.17 Problem 16 — The 0.1 + 0.2 problem

Evaluate:

```
1 0.1 + 0.2
2 0.1 + 0.2 == 0.3
```

Explain precisely why the equality comparison does not produce the mathematically expected result.

5.18 Problem 17 — Exact and inexact representations

Without relying only on the displayed decimal output, determine which of the following values can be represented exactly in binary floating-point form:

```
1 0.5
2 0.25
3 0.125
4 0.1
5 0.2
6 0.3
```

Relate your answer to their binary expansions.

5.19 Problem 18 — Floating-point equality

Suppose a Data Science pipeline produces:

```
1 expected = 0.3
2 observed = 0.1 + 0.2
```

Explain why:

```
1 expected == observed
```

is not an appropriate numerical comparison.

Implement an appropriate comparison using `math.isclose()` and justify the tolerance used.

5.20 Problem 19 — Machine epsilon

Use `sys.float_info.epsilon` to investigate the spacing between floating-point numbers near 1.0.

Compare:

```
1 1.0 + sys.float_info.epsilon
```

and:

```
1 1.0 + sys.float_info.epsilon / 2
```

with 1.0.

Explain the results in terms of finite floating-point precision.

5.21 Problem 20 — Precision depends on magnitude

Investigate the following expressions:

```
1 1e16 + 1
2 1e16 + 2
3 1e16 + 10
```

Determine which increments are distinguishable at this magnitude and explain why floating-point precision is not uniform across the real number line.

5.22 Problem 21 — Accumulation error

Compare the following two computations:

```
1 total = 0.0
2
3 for _ in range(10):
4     total += 0.1
```

and:

```
1 total = 10 * 0.1
```

Compare both results with 1.0 and explain any differences.

5.23 Problem 22 — Summation accuracy

Construct a sequence containing floating-point values with substantially different magnitudes.

Compare the results obtained using:

```
1 sum(values)
```

and:

```
1 math.fsum(values)
```

Investigate why the two approaches may produce different results.

5.24 Problem 23 — Catastrophic cancellation

Consider the function

$$f(x) = \sqrt{x+1} - \sqrt{x}.$$

Evaluate it numerically for increasingly large values of x .

Then derive an algebraically equivalent expression that avoids subtracting two nearly equal quantities and compare the numerical results.

Explain the difference in terms of catastrophic cancellation.

5.25 Problem 24 — Stable computation

The quadratic equation

$$ax^2 + bx + c = 0$$

has the standard solution

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Investigate numerically what happens when b^2 is much larger than $4ac$ and one root involves subtraction between nearly equal quantities.

Find an algebraically equivalent approach that improves numerical stability.

5.26 Problem 25 — Creating complex numbers

Create the complex number

$$z = 3 + 4i$$

in Python.

Determine:

- its type;
- its real part;
- its imaginary part;
- its complex conjugate;
- its magnitude.

Verify the magnitude analytically.

5.27 Problem 26 — Complex arithmetic

Given

$$z_1 = 3 + 4i, \quad z_2 = 1 - 2i,$$

compute by hand and then verify in Python:

$$z_1 + z_2, \quad z_1 - z_2, \quad z_1 z_2, \quad \frac{z_1}{z_2}.$$

5.28 Problem 27 — math versus cmath

Compare the behaviour of:

```
1 math.sqrt(-1)
```

and:


```
1 cmath.sqrt(-1)
```

Explain why the two modules behave differently.

5.29 Problem 28 — Cartesian and polar representations

For

$$z = 1 + i,$$

determine its magnitude and phase analytically.

Use `cmath.polar()` to verify the result and reconstruct the original number using `cmath.rect()`.

5.30 Problem 29 — Euler's identity

Evaluate:

```
1 cmath.exp(1j * cmath.pi) + 1
```

Mathematically, Euler's identity states:

$$e^{i\pi} + 1 = 0.$$

Explain why the computational result may not be exactly 0j.

5.31 Problem 30 — Predict the output

Without executing the code, determine the output and explain each result:

```
1 print(True + True)
2 print(True * 10)
3 print(False + 5)
4 print(isinstance(True, int))
```

5.32 Problem 31 — Equality across numeric types

Without executing the code, predict the result of:

```
1 1 == 1.0
2 1 == True
3 1.0 == True
4 0 == False
```

Explain the relationship between bool, int, and float that produces these results.

5.33 Problem 32 — A subtle dictionary key

Predict the result of:

```
1 data = {
2     True: "boolean",
3     1: "integer",
4     1.0: "float"
5 }
6
7 print(data)
8 print(len(data))
```

Explain the result in terms of equality and hashing.

5.34 Problem 33 — Floating-point threshold

A binary classifier returns a probability:

```
1 probability = 0.7
2 threshold = 0.7
```

A prediction is generated using:

```
1 prediction = probability >= threshold
```

Discuss whether floating-point representation creates a practical concern here and under what circumstances a tolerance-based comparison would or would not be appropriate.

5.35 Problem 34 — Comparing model metrics

Two evaluation procedures produce:

```
1 score_a = 0.8000000001
2 score_b = 0.8000000002
```

Determine whether it is meaningful to conclude that the second model is better solely from these values.

Discuss the distinction between:

- floating-point equality;
- numerical tolerance;
- statistical or practical significance.

5.36 Problem 35 — Large integer versus floating point

Consider:

```
1 x = 10**20
2 y = float(x)
```

Investigate whether:

```
1 x == y
```

is sufficient to conclude that float can exactly represent arbitrary integers of this magnitude.

Determine how integer precision differs between Python's int and float types.

5.37 Problem 36 — Numerical preprocessing

A preprocessing pipeline standardizes a feature using

$$z = \frac{x - \mu}{\sigma}.$$

Consider data for which x and μ are both very large and extremely close to one another.

Discuss the potential numerical issue in computing $x - \mu$ and identify the floating-point phenomenon involved.

5.38 Problem 37 — Probability normalization

Suppose a model produces a vector of probabilities whose floating-point sum is:

```
1 0.9999999999999999
```

rather than exactly 1.0.

Explain whether this necessarily indicates an error in the model or implementation, and describe how you would assess the result.

5.39 Problem 38 — Debugging a numerical comparison

The following assertion occasionally fails in a Machine Learning test suite:

```
1 assert calculated_score == expected_score
```

Both values are floating-point results produced through several numerical operations.

Explain why the assertion may be fragile and propose a more appropriate testing strategy.

5.40 Problem 39 — Reasoning about numerical stability

Two implementations compute the same mathematical quantity and produce nearly identical results on ordinary inputs. On very large or very small inputs, however, one implementation becomes inaccurate while the other remains stable.

Explain why mathematically equivalent expressions can behave differently in floating-point arithmetic and describe how you would investigate which implementation is numerically preferable.

5.41 Problem 40 — Choosing a numeric representation

For each of the following quantities, identify an appropriate Python representation and justify your choice:

- the number of observations in a dataset;
- a model accuracy score;
- a Boolean indicator of whether a model has converged;
- a monetary amount requiring exact decimal arithmetic;
- a complex-valued Fourier coefficient;

- an integer identifier containing 25 decimal digits.

Where the built-in int, float, bool, or complex types are not appropriate, identify what additional representation you would consider.